

ATANDA

Command Centre

Scenario Walkthroughs — F1 to F7, by Plan

Four real-shaped stories. One per subscriber tier. No jargon.

WHY THIS EXISTS

The pricing page tells you what each plan costs. The plain-English guide tells you what each plan is. This document does the third thing: it shows you what it actually looks like to USE each plan, step by step, by following one realistic person from their messy first idea all the way through to whatever finished artefact their tier supports.

HOW TO READ IT

Each persona walks through the engines they have access to, in order. For each engine you'll see four things: what they do (the action they take in the app), the input (what they hand the engine), what the engine does (in plain English), and the output (what they get back). At the end of each story there's a short "what they did with it" closing.

PLAN-TO-ENGINE QUICK REFERENCE

- EXPLORER (free): F1 + F2.
- PRACTITIONER (\$49/mo): F1 through F6.
- ARCHITECT (\$199/mo): F1 through F7, plus DE-SPC (evolve an existing SPC).
- INSTITUTION (custom): F1 through F7 at any scale, on-prem if needed.

ENGINE GLOSSARY

F1. Diagnose — the app reads your raw idea and tells you exactly what's vague, missing, or contradictory.

F2. Atomic Prompt — it rewrites your idea as a single, unambiguous prompt a model can't misread.

F3. Agent Birth Package — it grows that prompt into a CELL: a small, repeatable AI worker with a clear job.

F4. Micro PDD — it writes the short product spec for that worker (who, what, why, success criteria).

F5. SPC — it writes the full system blueprint: inputs, outputs, edge cases, rails, evaluation.

F6. ATLAS PDD — it formats the blueprint into the 4-part ATLAS deliverable (and recommends a VIBE DJ).

F7. MVP-PDD Certification — it compresses everything into the SPARTAN-certified MVP-PDD with a public verify URL you can share.

EXPLORER

Maya, a one-person newsletter writer

EXPLORER (free) · 5 sessions/day · F1 and F2 only

THE STORY SO FAR

Maya writes a weekly culture newsletter (3,400 subscribers). Every Sunday she pastes a long, rambling prompt into ChatGPT to draft her 'five-things-I-loved-this-week' section. The drafts are inconsistent — sometimes great, sometimes off-topic. She wants to fix the prompt itself.

WHAT SUCCESS LOOKS LIKE

Get a clean, repeatable prompt for her weekly 'five things' section that always returns the same shape of output.

F1 — Paste her current Sunday prompt into the F1 box and click Run.

INPUT

"Hey, can you give me five things I loved this week, make it sound like me, kinda fun and a bit nerdy, no boring stuff, maybe music or films or a book, and keep it short but not too short, also pick a theme if you can."

WHAT THE ENGINE DOES

Diagnose — the app reads your raw idea and tells you exactly what's vague, missing, or contradictory. It scores the prompt on the JCSE rubric (clarity, specificity, etc.) and lists every ambiguous phrase.

OUTPUT

JCSE score: 3.2 / 10. Flags: "sound like me" has no voice reference; "short but not too short" gives no length target; "pick a theme if you can" is optional and contradicts the fixed structure; no output format is specified.

WHAT HAPPENS NEXT

Maya is mildly embarrassed (she wrote this every Sunday for six months) but now she can see why some drafts went off.

F2 — Click 'Generate Atomic Prompt' on the F1 diagnosis. F2 rewrites her prompt as a single deterministic block.

INPUT

(F2 reads the F1 diagnosis automatically — Maya doesn't have to retype anything.)

WHAT THE ENGINE DOES

Atomic Prompt — it rewrites your idea as a single, unambiguous prompt a model can't misread. Every ambiguous phrase from F1 is replaced with a concrete constraint, and an output format is added.

OUTPUT

A reusable prompt that says: voice reference = 3 attached past issues; format = 5 items, each with a one-sentence pitch + one-sentence why-it-mattered, max 60 words each; theme = inferred from

the 5 items, stated as a single line at the top; no preamble, no sign-off.

WHAT HAPPENS NEXT

Maya saves it as her Sunday template. Next week's draft lands in the right shape on the first try. She has used 2 of her 5 daily sessions.

WHERE THIS LEAVES THEM

Maya does not need F3 onward — she isn't building an AI product, she's just making her own prompt better. The free EXPLORER tier was the right call. If she ever decides to package this prompt as a paid "weekly newsletter agent" service for other writers, that's when she'd move to PRACTITIONER and continue from F3.

PRACTITIONER

Daniel, a freelance developer building for a SaaS client

PRACTITIONER (\$49/mo) · 20 sessions/day · F1 through F6

THE STORY SO FAR

Daniel is a freelancer. A SaaS client has hired him to build a support-ticket triage agent that reads incoming Zendesk tickets and routes them to the right team (billing / bug / feature-request / abuse). The client wants a written blueprint they can hand to their internal dev team to maintain after Daniel is done.

WHAT SUCCESS LOOKS LIKE

Deliver a maintainable, documented triage agent — not just a working script — by the end of the two-week engagement.

F1 — Paste the client's brief ("we want AI to sort tickets") into F1.

INPUT

"Build us an AI thing that reads support tickets and figures out where they should go. We get maybe 200 tickets a day. Should be cheap and accurate."

WHAT THE ENGINE DOES

Diagnose — the app reads your raw idea and tells you exactly what's vague, missing, or contradictory.

OUTPUT

JCSE score 2.8 / 10. Flags: no list of categories; "accurate" is undefined; no SLA; no privacy/PII rules; no escalation path for ambiguous tickets; "cheap" has no budget.

F2 — Daniel meets with the client for 20 minutes to fill the gaps, then runs F2.

INPUT

F1 diagnosis + Daniel's notes from the client call.

WHAT THE ENGINE DOES

Atomic Prompt — it rewrites your idea as a single, unambiguous prompt a model can't misread.

OUTPUT

Atomic prompt: classify each ticket into exactly one of {billing, bug, feature, abuse, other}; return JSON with category, confidence (0-1), and a one-sentence reason; if confidence < 0.7 route to 'other' for human review.

F3 — Click 'Grow CELL'. F3 turns the atomic prompt into a Molecular Agent Birth Package.

INPUT

The F2 atomic prompt.

WHAT THE ENGINE DOES

Agent Birth Package — it grows that prompt into a CELL: a small, repeatable AI worker with a clear job.

OUTPUT

A CELL spec: input schema (ticket text + metadata), output schema (the JSON above), tools = none required, rails = strip PII before logging, fallback = 'other' on any parse failure. Estimated cost per 1k tickets included.

F4 — Click 'Write Micro PDD'.

INPUT

The CELL spec.

WHAT THE ENGINE DOES

Micro PDD — it writes the short product spec for that worker (who, what, why, success criteria).

OUTPUT

A one-page Micro PDD: problem statement, user (the support lead), success criterion (= 90% of tickets routed correctly without human touch, measured weekly), and what's explicitly out of scope (auto-replying, multi-language).

F5 — Click 'Build SPC'. This is the full system blueprint.

INPUT

Everything from F1-F4.

WHAT THE ENGINE DOES

SPC — it writes the full system blueprint: inputs, outputs, edge cases, rails, evaluation.

OUTPUT

A full SPC: input contract, output contract, every edge case Daniel can think of (empty ticket, attachment-only ticket, non-English, profanity, looks-like-spam), the eval set (50 historical tickets the client labelled), and the rollback plan if accuracy drops.

F6 — Click 'Render ATLAS PDD'. This is the deliverable Daniel hands to the client.

INPUT

The SPC from F5.

WHAT THE ENGINE DOES

ATLAS PDD — it formats the blueprint into the 4-part ATLAS deliverable (and recommends a VIBE DJ). The app also recommends a VIBE DJ — a stylistic 'voice' for how the agent should phrase its rationale text.

OUTPUT

A 4-part ATLAS PDD (Audience · Trigger · Logic · Artefact · Standard), plus a recommended VIBE DJ profile ('crisp internal-tool tone'). Exportable as PDF and JSON.

WHERE THIS LEAVES THEM

Daniel hands the ATLAS PDD + the working code to the client. The internal dev team can now maintain the agent because the blueprint is explicit. Daniel decides not to certify (F7) because the client owns the IP and prefers a private deliverable. He has used 6 sessions today — well under the 20/day cap. If the next client wants a public, verifiable certificate (e.g. to show investors), that's when Daniel will upgrade to ARCHITECT.

ARCHITECT

Priya, CTO of a 6-person legal-tech startup

ARCHITECT (\$199/mo) · 100 sessions/day · F1 through F7 + DE-SPC

THE STORY SO FAR

Priya's startup sells an AI assistant that pre-reviews NDAs for small businesses. She is about to raise a seed round, and her lead investor has asked for proof that the AI behaviour is auditable. She needs a public, verifiable blueprint — not just "trust us, we tested it".

WHAT SUCCESS LOOKS LIKE

Produce a SPARTAN-certified MVP-PDD with a public verify URL she can paste into her data room.

F1 — Paste the current product description into F1.

INPUT

"NDA Buddy reads an NDA and flags clauses that are unusual for a small business signing it."

WHAT THE ENGINE DOES

Diagnose — the app reads your raw idea and tells you exactly what's vague, missing, or contradictory.

OUTPUT

JCSE 4.5 / 10. Flags: "unusual" is undefined; "small business" has no jurisdiction; no list of clause types in scope; no failure mode for non-NDAs accidentally uploaded.

F2 — Resolve the flags with Priya's lead lawyer, then F2.

INPUT

F1 diagnosis + lawyer notes.

WHAT THE ENGINE DOES

Atomic Prompt — it rewrites your idea as a single, unambiguous prompt a model can't misread.

OUTPUT

Atomic prompt: detect document type first; if not an NDA, refuse with a fixed message; if NDA, classify each clause against a list of 14 known clause types and rate each on a 3-level unusual-for-SMB scale with a one-sentence explanation.

F3 — Grow the CELL.

INPUT

F2 atomic prompt.

WHAT THE ENGINE DOES

Agent Birth Package — it grows that prompt into a CELL: a small, repeatable AI worker with a clear job.

OUTPUT

CELL with input = PDF or DOCX upload, tool = clause-segmenter, rails = never give legal advice,

fallback = escalate to human reviewer for jurisdiction-specific clauses.

F4 — Write the Micro PDD.

INPUT

CELL.

WHAT THE ENGINE DOES

Micro PDD — it writes the short product spec for that worker (who, what, why, success criteria).

OUTPUT

Micro PDD: target user = SMB founder; success = founder understands risk in < 5 minutes; out of scope = redlining, negotiation drafting, jurisdictions outside UK/US/EU.

F5 — Build the full SPC. Because Priya already has a previous SPC (v0.3) from a hackathon, she uses DE-SPC instead of starting fresh.

INPUT

The previous SPC v0.3 + the new Micro PDD.

WHAT THE ENGINE DOES

DE-SPC (Digitally Evolved SPC) — gated by Priya's ASPE badge. It evolves the old SPC instead of rewriting from scratch, preserving the parts that already worked and updating only what changed.

OUTPUT

SPC v0.4 with a documented diff showing exactly what changed since v0.3.

F6 — Render the ATLAS PDD.

INPUT

SPC v0.4.

WHAT THE ENGINE DOES

ATLAS PDD — it formats the blueprint into the 4-part ATLAS deliverable (and recommends a VIBE DJ).

OUTPUT

ATLAS PDD with a recommended VIBE DJ profile ('cautious legal-explainer tone — never affirms, never advises').

F7 — Click 'Certify'. This is the moment Priya was building toward.

INPUT

ATLAS PDD from F6.

WHAT THE ENGINE DOES

MVP-PDD Certification — it compresses everything into the SPARTAN-certified MVP-PDD with a public verify URL you can share. The engine runs the SPARTAN compression and audit-grade math, then issues a certificate stored under a public URL.

OUTPUT

MVP-PDD signed and verifiable at <https://atanda.<your-domain>/verify/<cert-id>>. Priya drops the URL into her data room and her pitch deck.

WHERE THIS LEAVES THEM

Priya now has a public, third-party-verifiable artefact that proves her AI behaviour is bounded and audited. The investor closes the round two weeks later. She has used 8 sessions today (cap is 100). Next quarter she plans to certify the four other agents in the product suite the same way.

INSTITUTION

Westbridge Council — citizen-services team rolling out at scale

INSTITUTION (custom pricing) · Unlimited sessions · F1 through F7, on-prem deployment, dedicated support

THE STORY SO FAR

Westbridge Council wants to deploy an AI form-completion assistant inside their citizen-services portal. It must help residents fill in housing-benefit applications in plain English. Privacy, fairness, and FOI obligations mean the model must run inside the council's own perimeter, and every assisted form must be auditable.

WHAT SUCCESS LOOKS LIKE

Roll the assistant out to 22 frontline officers and 180,000 residents, with a certified blueprint for each of the 7 form types in scope.

F1 — The product lead (Asha) pastes the council's policy intent into F1. INPUT

"Help residents complete housing-benefit applications correctly so we reduce returned-for-correction rate, which is currently 38%."

WHAT THE ENGINE DOES

Diagnose — the app reads your raw idea and tells you exactly what's vague, missing, or contradictory.

OUTPUT

JCSE 5.1 / 10. Flags: 'correctly' undefined; no accessibility commitments; no language coverage; no handling for residents who lack documents; no rule for what the assistant must NOT do (e.g. estimate award amounts).

F2 — Asha runs F2 after resolving the flags with the policy team and the Equalities Lead.

INPUT

F1 diagnosis + policy notes + Equalities sign-off.

WHAT THE ENGINE DOES

Atomic Prompt — it rewrites your idea as a single, unambiguous prompt a model can't misread.

OUTPUT

Atomic prompt: explain each field in plain English at reading age 11; never estimate award amounts; flag missing documents and link to the upload step; offer to switch to large text or screen-reader mode.

F3 — Grow the CELL.

INPUT

F2 atomic prompt.

WHAT THE ENGINE DOES

Agent Birth Package — it grows that prompt into a CELL: a small, repeatable AI worker with a clear job.

OUTPUT

CELL: input = current form field + resident's question; tool = council policy retrieval index (on-prem vector store); rails = no PII echoed back, no advice on award value, full transcript persisted for FOI.

F4 — Write the Micro PDD.

INPUT

CELL.

WHAT THE ENGINE DOES

Micro PDD — it writes the short product spec for that worker (who, what, why, success criteria).

OUTPUT

Micro PDD per form type (7 forms × 1 each), each with its own success metric and out-of-scope list.

F5 — Build the SPCs. Because the 7 forms share most of their behaviour, the team uses DE-SPC after building the first one — every later SPC is evolved from the first.

INPUT

Form 1 SPC v1.0 !' evolved into Forms 2-7.

WHAT THE ENGINE DOES

F5 for the first form; DE-SPC for forms 2 through 7. Each evolution is documented with a clean diff.

OUTPUT

7 SPCs sharing a common base, with explicit per-form deviations.

F6 — Render all 7 ATLAS PDDs.

INPUT

The 7 SPCs.

WHAT THE ENGINE DOES

ATLAS PDD — it formats the blueprint into the 4-part ATLAS deliverable (and recommends a VIBE DJ).

OUTPUT

7 ATLAS PDDs, each with a shared VIBE DJ profile ('warm, plain-English, never-condescending public-service tone').

F7 — Certify all 7 in one batch.

INPUT

The 7 ATLAS PDDs.

WHAT THE ENGINE DOES

MVP-PDD Certification — it compresses everything into the SPARTAN-certified MVP-PDD with a public verify URL you can share.

OUTPUT

7 MVP-PDDs, each with a public verify URL that the council publishes on its transparency page. The transparency page itself links to all 7 — anyone (a resident, a journalist, an auditor) can see the bounded behaviour of every assistant.

WHERE THIS LEAVES THEM

Six months after rollout, returned-for-correction rate drops from 38% to 14%. The transparency page receives one FOI request, which is answered by sending the link to the relevant verify URL. Asha presents the case study at LocalGovCamp. Because the council is on INSTITUTION, the same blueprint approach now scales to the next 9 form types without new procurement.

Cross-tier reflections

THE SAME IDEA GROWS UP WITH YOU

Notice that Maya, Daniel, Priya, and Asha all start the same way — they paste a messy human description of what they want into F1. The difference between the tiers is not the start, it's the finish: how far down the F-chain you can take that same idea before you run out of road.

YOU DON'T ALWAYS NEED TO REACH F7

Maya stops at F2 because her artefact is a personal prompt template. Daniel stops at F6 because his client wants a private deliverable. Only Priya and Asha need F7, because their artefact is a public, verifiable claim about how an AI behaves. Pick the tier that matches your finish line — not the most expensive one.

DE-SPC IS THE QUIET SUPERPOWER

Both Priya and Asha use DE-SPC (Digitally Evolved SPC) — once you've built one good SPC, you almost never start from scratch again. That's why ARCHITECT and INSTITUTION users compound output over time and PRACTITIONER users sometimes consider upgrading after a few months.

THE VERIFY URL IS THE RECEIPT

Every F7 certificate produces a public URL. That URL is the difference between "I built an AI thing" and "here is a verifiable description of what my AI thing will and will not do, signed and dated". For investors, regulators, FOI officers, and procurement teams, that distinction is the whole point.